

AI-Powered System for Ripeness Detection and Post-Harvest Storage Advice

T. Manasa Veena¹, Shaik Hasane Ahammad^{1*}, Shaik Mahimud², Amudala Sulochana Kumari³, Kodali Suchitra Kamala⁴, Chimata Phanindra Babu⁵

¹Department of ECE, NRI Institute of Technology, Pothavarapadu, Vijayawada-521212.

^{1*}Department of ECE, Koneru Lakshmaiah Education Foundation, Vaddeswaram, India-522302.

E-mails: manasaveena.t@nriit.edu.in, ahammadklu@gmail.com

^{2,3,4,5}Department of AI&ML, NRI Institute of Technology, Pothavarapadu, Vijayawada-521212.

E-mails: mahimudshaik666@gmail.com, sulochanakumaria@gmail.com, kodalisuchitrakamala@gmail.com, phanichimata21@gmail.com

Corresponding Author: T.Manasa Veena (manasaveena.t@nriit.edu.in)

Abstract— Based on the situations of the farmers who are facing a major challenge in the agricultural sector for post-harvesting of the fruits and vegetables. This problem majorly occurs due to the improper detection of ripeness and unsuitable storage practices. Traditionally the manual methods to estimate ripeness are often becomes inaccurate, time-consuming, and subjective. To address this To address these issues, this project presents a AI-Powered System which uses to processes images and utilizes machine learning techniques to detect ripeness automatically of a fruit or vegetable and provide post-harvesting storage recommendations to the farmers and the vendors. In previous method they have used YOLOv5, YOLOv6, YOLOv7 and SSD-MobileNetv1 for detecting fruit and classifying their ripeness in real-world images. And used to explain the manual image dataset which consist of strawberries and avocados images under natural conditions with four different stages like- Unripe, Partially Ripe, Ripe, Rotten. In this dataset the images are naturally captured under the lighting and featured varied backgrounds, occlusions for reflecting real harvesting environments and test robustness and the dataset is bounded by boxes around fruits and corresponding ripeness stage labels the dataset can be available in the Mendeley. The purpose of dataset enables benchmarking of object detection model (YOLOv5, YOLOv6, YOLOv7, and SSD-MobileNetv1) for multi-class fruit ripeness classification, rather than just binary ripe/unripe classification. In previous it have been used MobileNetV2 which is lightweight convolutional neural network designed for efficient feature extraction so based on it in project we are using EfficientNetB0 which is similar to it where as both models aims to achieve high accuracy with reduced computational cost, making them suitable for real-time agricultural image analysis and resource constrained environments. The MobileNetV2 that is used in previous is highly cost-efficient, very low Parameters count (~3-4 million parameters), fast interface time, lower training cost, Uses depthwise separable convolutions, Consumes less GPU/CPU power Shorter training time, Lower accuracy trade-off, Well-suited for mobile and embedded systems, Ideal for large-scale deployment. With the help of MobileNetV2 in the previous method it get overall accuracy of 99.3% and as of now using EfficientNetB0 the project getting the overall accuracy of 99.4%. **Post-harvest losses pose a significant challenge to the global agricultural industry and are primarily caused by people's incapacity to determine the ripeness of crops and the lack of scientific storage guidelines. In order to address these issues, this study proposes a combined computer vision pipeline that can automatically determine the ripeness of fruit and provide storage guidance. For the first time, the system uses YOLO-based architectures to locate and distinguish objects. It accomplishes this using a sizable dataset of 1,200 images from the Mendeley Data repository of oranges, mangoes, bananas, and apples. The**

primary novel concept is to classify ripeness into multiple classes using an improved EfficientNetB0 architecture. We selected EfficientNetB0 due to its superior scaling coefficient and its enhanced speed compared to other convolutional neural networks (CNNs). The experiment was based on splitting the dataset into training and testing sets in an 80/20 ratio and using a wide range of data augmentation methods, such as rotation, scaling, and changing the brightness. The proposed system was able to classify ripeness with an impressive 99.4% accuracy. The system not only detects ripeness, but it also suggests where to store the food based on the stage of ripeness it finds. This is a great way to make food last longer and cut down on waste.

Keywords- EfficientNetB0, Fruit Ripeness Detection, CNN, Deep Learning, Object Detection, Computer Vision, Multiple Languages

I. INTRODUCTION

This paper aims to systematically review and compare the efficacy of various machine learning and deep learning methodologies applied to fruit ripeness detection, addressing the critical need for efficient and accurate assessment in agricultural automation (Salim & Mohammed, 2024) (Zhou et al., 2025). Specifically, the focus will be on deep learning approaches that have demonstrated superior accuracy compared to other methods in vision-based systems for fruit ripeness detection (Zhang, 2023; Zhou et al., 2025). Despite these advancements, ongoing challenges persist in achieving high accuracy in complex scenarios involving variable lighting, fruit sizes, and occlusions (Zhang, 2023). This is where the scalability and end-to-end learning capabilities of deep learning models, particularly CNN's, become crucial, allowing them to adapt to diverse datasets and eliminate the need for manual preprocessing and feature engineering (Goh et al., 2024). Moreover, deep learning architectures, such as improved DenseNet, effectively manage challenges like dataset imbalance and sparse data through innovative loss functions and structured sparse operations, optimizing performance in real-world agricultural settings (Vo et al., 2024). Among recent fruit ripeness detection studies, YOLO and its optimized variants such as YOLOv5, YOLOv8, YOLOv9c, CES-YOLO (custom YOLO), and YOLO + Transformer have been extensively utilized due to their real-time detection capability, single-stage object detection, and high accuracy with low inference time (Goh et al., 2024; Salim & Mohammed, 2024; Zhou et al., 2025). Furthermore, their suitability for edge devices and field

deployment, coupled with their ability to effectively handle multi-class ripeness stages, makes them highly attractive for precision agriculture applications (Wang et al., 2025; Xiao et al., 2023). Beyond YOLO, CNN-based lightweight models like MobileNet, Lightweight CNN, and Mask R-CNN are also employed, primarily due to their low computational cost and efficiency on mobile and IoT platforms (Singh et al., 2025). Conversely, Transformer-based models, including RT-DETR, Vision Transformer, and Attention-based YOLO, represent an emerging trend, demonstrating enhanced global context understanding and improved accuracy in complex backgrounds, despite their comparatively higher computational demands (Zhang, 2023).

Among recent fruit ripeness detection studies, YOLO(You Only Look Once) and its optimized variants used like YOLOv5, YOLOv8, YOLOv9c, CES-YOLO(custom YOLO), YOLO + Transformer (Hybrid) due to Real-time detection capability, Single-stage object detection, High accuracy with low inference time, Suitable for edge devices and field deployment, Handles multi-class ripeness stages effectively. And then CNN-based Lightweight Models used like MobileNet, Lightweight CNN, Mask R-CNN due to Low computational cost, Efficient on the mobile and IoT platforms. And then Transformer-based models (Emerging trend) used like RT-DETR, Vision Transformer, Attention-based YOLO due to better global context understanding, improved accuracy in complex backgrounds, still computationally heavy compared to YOLO. And then Traditional ML Models used like SVM, Random Forest, KNN due to it require manual feature extraction, Lower accuracy on image-based tasks.

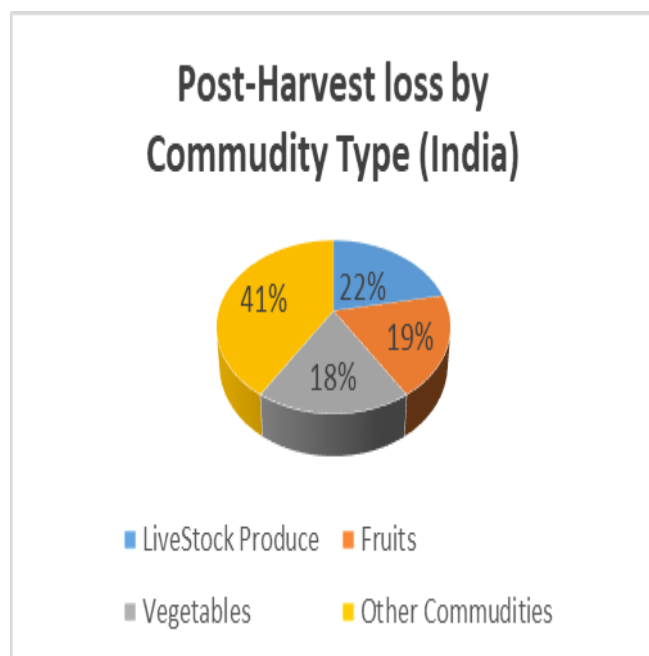


Figure 1: Post-Harvesting loss by Commodity type-India, 2022

The above image is showing the loss percentage in Post-Harvesting by the commodity type India in 2022 with the following percentages like Live Stock Products have 41%, Fruits have 19%, vegetables have 18%, and other commodities have 22%. To contribute the losses of fruits

nearly ₹29,000 crore annually in India. By using the SSD-MobileNetV2 (lightweight backbone + detector), YOLOv5/YOLOv6/YOLOv7(object detection models).

The characters that is used in the each model are fast inference and good accuracy, widely used in many real-world vision tasks, Learns features at different scales to localize objects of varying sizes, focuses on accuracy, speed, balancing the performance, excellent precision and feature learning for detecting fine difference between ripeness stages, slightly different model architecture which gets modified by comparing the YOLOv5 and YOLOv6, Designed for speed and efficiency, lower accuracy compared to YOLO model for SSD-MobileNetV2.

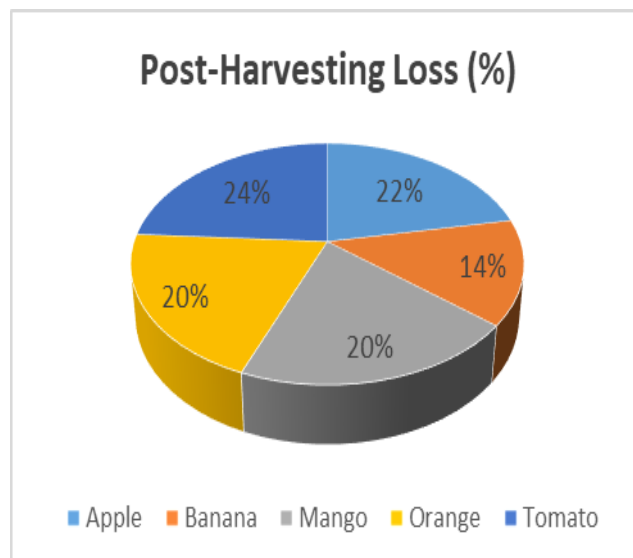


Figure 2: The above pie chart describe the loss of fruits that is having loss in Post-Harvesting.

The above image is showing the loss percentage in Post-Harvesting for the few fruits with the following percentages like Apple have 22%, Banana have 14%, Mango have 20%, Orange have 20%, tomato have 24%. The pie-chart percentages were derived by normalizing the average post-harvest loss values for only the fruits like-Apple, Banana, Mango, Orange, Tomato to represent their relative contribution to total losses within the group. Even though the actual loss for the above fruits are 11% for apple, 7% for banana, 10% for mango, 10% for orange, and 12% for tomato, these values are converted into proportional shares when it is summed to 100% for visualization purposes. this type of normalization highlights the comparative impact of each fruit on overall post-harvest losses rather than absolute loss rates, with tomato and apple where these are contributing the highest shares due to their habitability and sensitive handling. It is taken between years of 2020-2024, which is primarily focusing on the India and similar tropical agricultural supply chain region. These averaged values are subsequently normalized to 100% to illustrate the relative contribution of each fruit to show the overall post-harvesting losses rather than depict losses for different specific years. **Increasingly, the agricultural sector is employing artificial intelligence to address logistical challenges. Knowing exactly what stage of ripeness the fruit is in—unripe, ripe, or overripe—is very important for**

figuring out the best time to transport and sell it. Current manual methods are prone to subjectivity and errors, leading to significant spoilage. There is still a need for a system that combines high-accuracy classification with useful information after harvesting, even though other studies have used different Deep Learning models.

II. LITERATURE SURVEY

The CNN-models that are used in the previous papers are YOLO(You Only Look Once with Darknet / CSPDarknet backbone), SSD(Single Shot Detector With VGG / MobileNet backbone, Lightweight CNN (custom CNN / MobileNet-based CNN), MobileNet, Depthwise Separable CNN, CES-YOLO (YOLO with optimized CNN backbone), YOLOv8 CNN backbone, CNN + Attention Modules, YOLOv9c CNN backbone, Lightweight Mask R-CNN, ResNet-lite backbone, CNN Backbone within RT-DETR (ResNet / MobileNet), ResNet-based CNN backbone, Feature Pyramid CNN, VGG (Visual Geometry Group), ResNet, Inception, EfficientNet.

YOLO is used provides real-time detection with high accuracy, SSD is used to offers a good trade-off between speed and precision, these both are combined to form a single-stage CNN detectors, ideal for agricultural environments. The Lightweight CNN Model is used to suitable for IoT and low-power devices with low computational cost, and also efficient for binary or multi-stage ripeness classification. The MobileNet, Depthwise Separable CNN models are used due to strawberries are small and delicate, MobileNet reduces parameters and FLOPs (Floating Point Operations), and suitable for real-time deployment. The CES-YOLO is used due to Blueberries are small and densely clustered, CES-YOLO is used to enhance feature extraction at small scales, designed for the edge devices. YOLOv8 CNN backbone with Attention Modules are used due to CNN excels at local feature extraction, Attention is improved at global context understanding, Helps in partially hidden and complex backgrounds. YOLOv9c CNN backbone is used due to it improves features reuse and gradient flow, handles difficult lighting conditions and object hiding parts of other objects, optimized for multi-factor agricultural scenes. The lightweight Mask R-CNN ResNet-lite backbone is used due to simultaneous working of ripeness detection and pest detection, Instance segmentation is requiring pixel-level accuracy, Lightweight CNN reduces computations in model. The CNN backbone within RT-DETR is used due to CNN works efficiently to extracts visual features, RT-DETR improves detection accuracy, Lightweight CNN is to enabling faster inference. The ResNet-based CNN backbone, Feature Pyramid CNN are used due to Strong feature extraction capability, Robust for varying fruit shapes and lighting of a fruit, CNN integrates well with transformers. The VGG, ResNet, Inception, EfficientNet are used due to CNN's dominate image-based ripeness detection, Deep CNN's to out perform traditional ML techniques, Transfer Learning will improves performance with small datasets.

Recent developments in fruit ripeness detection techniques have been based on different variations of YOLO, such as YOLOv8, YOLOv9, and instance

segmentation models. However, many of these studies are based on fruit ripeness detection, but none of them consider providing the necessary storage metadata, which is essential for farmers and retailers. This section discusses 14 key papers related to the transition from traditional CNNs to transformer-based and lightweight models for agricultural applications.

YOLO, SSD is used for CNN backbone will extracts spatial and texture features of the fruits, YOLO will predicts bounding boxes and ripeness class in one forward pass, SSD will performs multi-scale features detection using CNN feature maps, and outputs the classified fruits into unripe, ripe, and overripe. Lightweight CNN is used for CNN will extracts color, texture, and surface features, and combined with sensor data of temperature and humidity, CNN will output the fused with ML classifiers for spoilage detection a fruit. MobileNet, Depthwise Separable CNN is used for CNN is a backbone to processes high-resolution of images, Depthwise convolution will captures fine-grained ripeness cues, In Final layers is classifying strawberries into ripeness stages. CES-YOLO is used for CNN backbone will extract multi-scale features, Feature pyramid network will improves small-object detection, and it will outputs the predicted berry location and simultaneously ripeness stage. YOLOv8 CNN backbone, CNN attention modules are used for CNN will extract spatial features, Attention Modules will refine CNN feature maps YOLOv8 head will performs ripeness classification and localization. CNN backbone YOLOv9c is used for CNN backbone will extracts color-texture patterns, will improve the neck structure with enhanced feature fusion, Detection head will outputs ripeness stage with confidence. Lightweight Mask R-CNN, ResNet-lite backbone is used for CNN backbone will extracts hierarchical features, Mask head will segments fruit regions and pest regions, Classification head will identifies ripeness stage of a fruit. CNN backbone within RT-DETR is used for CNN will extracts feature maps, Transformer layers will refine class relationships, and output layer will predicts ripeness class and bounding box. Feature Pyramid CNN, ResNet-based CNN backbone is used for CNN will extracts low and high-level features, Features will passed to transformer encoder, Detection head will outputs ripeness classification. VGG, ResNet, Inception, EfficientNet is used for CNN's to act as feature extractors, Fully connected layers will classify ripeness stages, and will compare based on accuracy, speed, and robustness.

III. EXISTING METHODOLOGY

The existing methodology focus on real-time multi-class fruit ripeness detection by integrating deep convolutional feature extraction with a single-shot object detection frameworks. Using the methodology of the existing one the system have been designed to identify fruits automatically present in an image and simultaneously classifying ripeness stages such as Unripe, Ripe, or Overripe. Initially the image is captured under real-world agricultural conditions it is taken as input to send for processing the steps by resizing, normalizing and data augmentation to enhance robustness against different variations in lighting, orientation, and background. After preprocessing the feature extraction is performed using lightweight convolutional neural network

(CNN) which works as backbones, enabling efficient learning to texture, color, and shape-related to ripeness cues. The current state-of-the-art techniques heavily rely on object detection architectures such as YOLOv5, YOLOv8, and SSD (Single Shot Multibox Detector). These models perform well in detecting the position of the fruit in an image, also known as bounding box detection, as well as segmentation at the pixel level. However, they may not perform well in the extraction of the features required in the differentiation of the ripeness gradient in varying lighting conditions. Then the extracted features are then passed to object detection models, namely YOLO and SSD-MobileNetV2, which is used to predict bounding boxes along with corresponding ripeness class labels in a single forward pass. YOLO is used for its high detection speed and real-time capability, while SSD-MobileNetV2 is used for its computational efficiency and suitability for edge-device deployment. The use of MobileNetV2 works as a backbone to significantly reduce model complexity while maintaining reliable accuracy. The system will perform to evaluate to use the standard metrics such as accuracy, recall, and mean average precision, and demonstrating its effectiveness to have practical agricultural applications.

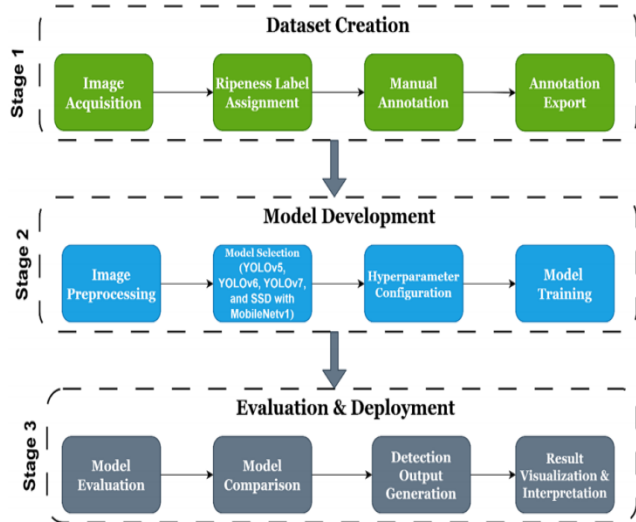


Figure 3: The image is the proposed frameworks that works in the existing method.

According to the first stage is Dataset Creation, second stage is Model Development, Third stage is Evaluation & Deployment. The description for each stage is described as following:

A. Dataset Creation

The first stage of the proposed system deals with the creation of the dataset, which forms the basis of a reliable fruit ripeness recognition process. Images are captured under real-world and lab situations, considering a wide array of farm scenarios, and accordingly marked Unripe, Ripe, and Overripe categories. Annotation of the images has been carried out manually, involving identifying the region of fruit. The following are used in the Data Creation:

- *Image Acquisition:* After obtaining the image, ripeness labeling is done to classify each fruit based on its level of ripeness. Fruits are categorized as Unripe, Ripe, and Overripe based on their appearance using color, texture, and surface details with the expertise of agriculturists. Labeling plays a critical

role as it provides ground truth for training a deep learning model to learn ripeness features effectively.

- *Ripeness Label Assignment:* After obtaining the image, ripeness labeling is done to classify each fruit based on its level of ripeness. Fruits are categorized as Unripe, Ripe, and Overripe based on their appearance using color, texture, and surface details with the expertise of agriculturists. Labeling plays a critical role as it provides ground truth for training a deep learning model to learn ripeness features effectively.
- *Manual Annotation:* After the assignment of ripeness labels, the annotation process is done by hand to identify the location of fruits in each image. Boxes are encircled around each fruit and connected with their respective ripeness labels. These annotations are quite crucial for training object detection models like YOLO and SSD because they enhance accuracy levels while reducing false detection rates.
- *Annotation Export:* The final process within Stage 1 is the export of annotations, whereby all the labels and the bounding box annotations are exported into formats that support the target deep learning environments. Annotations are therefore exported into standardized formats, including the YOLO text format or formats for SSD environments, thereby ensuring an efficient and error-free training process as the dataset is prepared for further processing and development of the model.

B. Model Deployment

Stage two involves model development. The prepared dataset will be utilized to develop models that detect fruits and classify their ripeness. Techniques of image processing are employed in preprocessing images by resizing them to ensure robustness. Normalization techniques are similarly employed to facilitate robustness of models trained by utilizing YOLO or SSD models supported by light convolutional neural networks. Hyperparameters are optimized based on recommended values to ensure models achieve convergence. The following are used in the Model Development:

- Image Preprocessing :* Beginning the development of the model involves the preprocessing of images to ensure that the data is adequately prepared for effective deep learning. Images are resized to a uniform size, normalized to ensure stability during the learning processes, and further enhanced through augmentation techniques including rotation, flipping, scaling, and brightness adjustments. All these measures ensure that the data is rich and that the model is not sensitive to variations in lighting, background, and fruit orientation.
- Model Selection:* Following the image processing, appropriate models for the detection of fruit ripeness are chosen. These include the latest models in the field, including YOLOv5, YOLOv6, YOLOv7, and the SSD models and use lightweight CNN models such as MobileNet. The reason for choosing these

models is their ability to detect objects in real-time and their accuracy.

- c. *Hyperparameter Configuration*: Next, the configuration of the hyper-parameters takes place to improve the training process. Learning rate, batch size, epoch numbers, choice of the optimizers, and anchor box parameters are some crucial hyper-parameters that need to be properly configured to avoid both overfitting and underfitting. This phase has a great impact on improving the accuracy of the fruit detectors
- d. *Model Training*: This represents the last process in Stage 2, where the trained detection models are trained based on the annotated dataset. During this process, the models are trained to learn the discriminative features that can be achieved by minimizing the collective loss on the classification, localization, and confidence of the models over various epochs. This trained model can detect fruits effectively, both in localization as well as the stages of ripeness.

C. Evaluation & Deployment:

The last stage involves the evaluation and application of the trained models for usability purposes. The performance metrics for evaluation include accuracy, precision, recall, F1 measure, and mean average precision; and based on these criteria, the best model is selected taking into consideration accuracy, speed, and efficiency. The selected model is then able to produce output for detection with boxes and ripeness state, which is displayed in a user-friendly interface for application in real-time agricultural and post-harvest scenarios. The following are used in the Evaluation and Deployment:

a. Model Evaluation

Performance evaluation on this trained detection model is measured by standard metrics such as precision, recall, F1-score, mean Average Precision-mAP, and confidence thresholds on a separate test dataset. The step validates the model's ability to correctly localize fruits and their stages of ripeness with higher accuracy under unseen conditions. In addition, this allows the identification of strengths and weaknesses, ensuring reliability for further real-world deployment.

b. Model Comparison

After that, the selected models will be individually evaluated to compare and identify the best models. Models like YOLOv5, YOLOv6, YOLOv7, and SSD models using MobileNet will be compared based on their speed, accuracy, computational complexity, and memory requirements. Based on that, an optimal model can be selected, providing the best trade-off between the performance of the object detection task and its feasibility of implementation.

c. Detection Output Generation

After model comparison, the best performing model is picked for deployment by generating detection outputs for input images or live camera feeds. It will be producing bounding boxes around the detected fruits, along with predicted ripeness labels and confidence

scores. This step translates the raw model predictions into clear and interpretable results, enabling the automated and practical identification of fruit ripeness in real agricultural environments.

d. Result Visualization and Interpretation

The final step involves visualization of the detection outcomes for end users through the placement of bound boxes, ripeness, and confidence onto the original image. The visualization makes decision-making easy in the context of post-harvest handling, storage, and quality evaluation. The approach increases the level of transparency in decision-making due to simplified prediction outcomes.

D. Equations & Explanation

a. YOLO/SSD Localization (Coordinate) Loss:

This loss penalizes incorrect predictions of the center position of fruit.

$$\lambda_{count} \sum_{i=0}^{s^2} \sum_{j=0}^B 1_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad (1)$$

Where as s^2 : Grid size into which the image is divided B: Number of bounding boxes per grid cell 1_{ij}^{obj} : Indicator function (1 if an object is present in box j of cell I, else 0) x_i, y_i : Ground truth center coordinates $\hat{x}_i, \hat{y}_i$: Predicted center coordinates λ_{count} : Weight to emphasize localization accuracy.

b. Bounding Box Dimension Loss: This ensures accurate size estimation of fruits in the image

$$\lambda_{count} \sum_{i=0}^{s^2} \sum_{j=0}^B 1_{ij}^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \quad (2)$$

Whereas: w_i, h_i : Ground truth width and height, $\hat{w}_i, \hat{h}_i$: Predicted width and height, Square root reduces sensitivity to large boxes.

c. Class Probability Loss (Ripeness Classification):

It ensures correct classification of fruit ripeness once the object is detected.

$$\sum_{i=0}^{s^2} \sum_{c \in \text{classes}} 1_{ij}^{obj} (p_i(c) - \hat{p}_i(c))^2 \quad (3)$$

Where as c: Ripeness classes (Unripe, Ripe, Overripe), $p_i(c)$: Ground truth probability, $\hat{p}_i(c)$: Predicted probability.

d. Object Confidence Loss (Object Present): It improves detection reliability by reinforcing true positives

$$\sum_{i=0}^{s^2} \sum_{j=0}^B 1_{ij}^{obj} (C_i - \hat{C}_i)^2 \quad (4)$$

Where as: C_i : Ground truth confidence (1 if object exists)

$\hat{C}_i$: Predicted confidence score.

e. Softmax Function: $f(s)_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$

f. Cross-Entropy Loss:

$$\text{Cross Entropy} = - \sum_i^N t_i \log(f(s)_i) \quad (6)$$

Where as t_i : Ground truth label, $f(s)_i$: Softmax probability

g. Mean Average Precision (mAP): mAP is the primary object detection metric, combining localization and classification accuracy.

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (7)$$

Whereas AP_i : Average Precision for class I, N: Number of classes.

h. Intersection over Union (IoU)

$$IoU = \frac{B_p \cap B_{gt}}{B_p \cup B_{gt}} \quad (8)$$

Where as, B_p : Predicted bounding box, B_{gt} : Ground truth bounding box.

- i. **Precision:** Indicates how many detected fruits are correctly identified.

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

- j. **Recall:** Measures how many actual fruits were successfully detected

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

- k. **F1-Score:** Harmonic mean of precision and recall, balancing false positives and false negatives.

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

IV. PROPOSED METHODOLOGY

Based on the existing methodology in this section giving a detailed explanation for the methodology that being used in this paper that is about EfficientNetB0. It is going to act as the visual reasoning backbone of the proposed system for ripeness detection which is responsible to transform the raw fruit images into meaningful ripeness related feature.

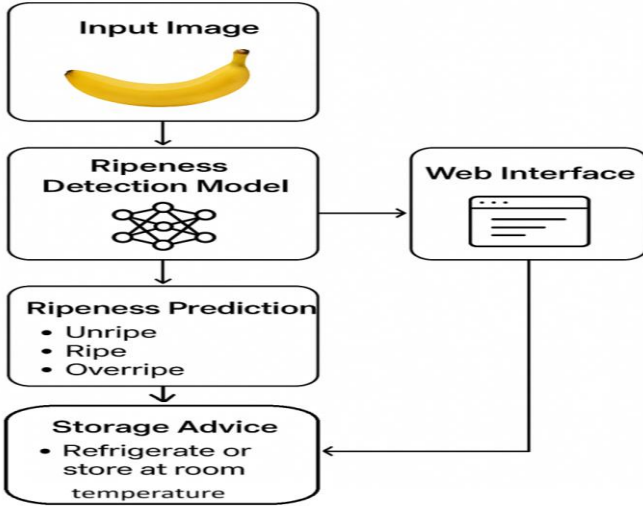


Figure 4: The image is block diagram of the system working

A. Input Image:

The input image acquisition module serves as the entry point for the ripeness detection process, whereby real images of fruits, acquired from mobile phones, digital cameras, or from files, can be provided by the users. In this case, since there are no restrictions regarding illumination, background, and quality of the images, this process can effectively suit real farm conditions.

B. Ripeness Detection Model(Core Intelligence Unit):

The Detection Model is essentially the brain of the entire system as it works on analyzing the fruit images uploaded onto the system for ripeness. The model uses EfficientNetB0 for the purpose of feature extraction and is used for standardizing the images and analyzing features such as color and texture.

C. Ripeness Prediction Output:

In this respect, the model delivers the ultimate ripeness classification, which can either be Unripe,

Ripe, or Overripe, based on the confidence levels of each class instead of fixed thresholds. Unripe means the fruit is immature, Ripe means the fruit is perfectly ripe for harvest or eating, and Overripe means the fruit has started to degenerate in its ripeness due to the natural fruit-ripening process.

D. Storage Advice Generation (Decision Support Layer):

This level of translation provides an interpretation of the maturity degree into post-harvest handling and storage. The fruits that are unripe can be stored at room temperature for ripening, ripe fruits can be stored for short-term refrigerated storage or for immediate consumption, while overripe fruits can be stored in the refrigerator or processed

E. Web Interface (Human-System Interaction Layer):

It acts as the interface between a human and the system by allowing users to upload images and, in turn, view predictions and storage advice. It represents clear presentation of the ripeness labels and levels of confidence and recommendations, hiding AI complexities for better access and building user trust in the system outputs.

F. Feedback Loop Between Model and Interface:

The feedback mechanism brings the model together with the web interface and allows the AI system to work as a holistic decision-making assistant. The predictions are translated into concise storage recommendations, thereby establishing a connection between artificial intelligence output and user understanding.

The system being proposed follows a pipeline architecture consisting of:

A. Detection and Annotation: The system utilizes a YOLO-based segmentation technique to separate the fruit from the rest of the image's background. The dataset, which was downloaded from Mendeley, was manually annotated for four types of fruits at three stages of ripeness.

B. Classification using EfficientNetB0: The isolated fruit is then passed to a classification model, EfficientNetB0, which is a model that utilizes compound scaling to ensure maximum accuracy with a small number of parameters.

C. Storage Recommendation Engine: This module, based on the classification results, provides a logical recommendation for the fruit's storage, e.g., "Store at 4°C for 5 days" for ripe apples, "Immediate processing" for overripe mangoes, etc.

From the block diagram provided, it can be noted that EfficientNetB0 takes the resized and normalized input image to extract the features related to ripeness. While the stem convolutional layer in EfficientNetB0 captures low-level characteristics such as color and edges, the subsequent Mobile Inverted Bottleneck layers are responsible for the extraction of higher-level characteristics like texture, bruise, and uniformity. In this architecture, the higher levels of the pre-trained EfficientNetB0 model are fine-tuned to specialize in ripeness features, holding onto the general image knowledge learned for the ImageNet dataset. Redundancy removal by MBConv layers, global average

pooling to encode ripeness features compactly, and density layers to generate classifications along with confidence values result in the accurate, efficient, and scalable nature of the EfficientNetB0 model. The clear explanation of the EfficientNetB0 working is shown with below architecture:

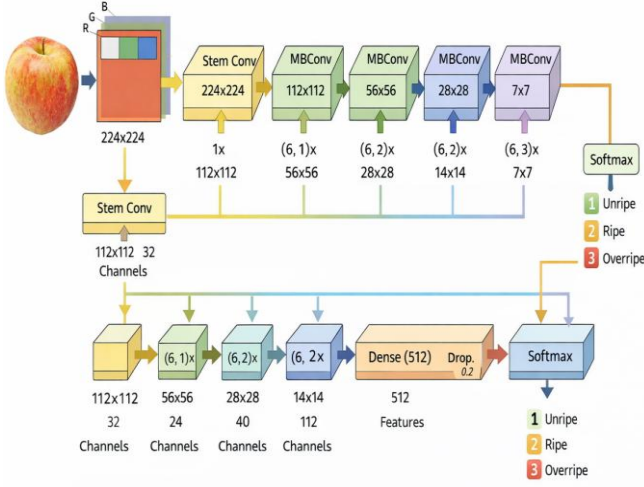


Figure 5: The architecture of the EfficientNetB0 which is used in the model training.

The steps that are used in the model training by using EfficientNetB0 is described in the below with certain steps as follows:

A.

Input Fruit Image ($RGB \rightarrow 224 \times 224$):

The normalization of the fruit image is then accomplished through this step in order to convert the image into a standardized size of 224×224 and then split the image into its red, blue, and green components, as the red is associated with the maturity of the fruit, the green indicates the decay associated with chlorophyll, and the blue indicates the reflectance and bruising.

B.

tem Convolution (Initial Visual Encoding):

It is the first stage where the input image is transformed into a feature map that measures 112×112 pixels and comprises 32 channels, and the process extracts raw features of the visual world, including edges, colors, and brightness variations. It also normalizes lighting and image quality, allowing accurate features to be extracted that are used by the EfficientNet model for ripeness evaluation.

C.

BConv Block - Stage 1 ($112 \times 112 \rightarrow 56 \times 56$):

This leads to a feature map size of 56×56 pixels with an increase in channel depth to 24, thereby improving feature extraction capabilities. The model is able to extract pre-ripeness signs such as color variations and surface patterns of fruits and vegetables, where a six-fold expansion is applied to examine high-dimensional features before compressing important pre-ripeness features for effective detection.

D.

BConv Block - Stage 2 ($56 \times 56 \rightarrow 28 \times 28$):

The feature map now is 28×28 with 40 channels, allowing the model to grasp more abstract features of the ripeness, such as saturation, texture, and patch-level cues. While the spatial details are reduced at this stage, greater semantic

resolution of the network on ripeness would enhance the sensitivity toward meaningful patterns rather than the exact positions of each pixel.

E.

BConv Block - Stage 3 ($28 \times 28 \rightarrow 14 \times 14$):

By this stage, the feature map is down to 14×14 with 112 channels; hence, the model will be able to capture complex signs of ripeness such as texture deterioration, dullness, and loss of shine. This is the trade-off between spatial detail and semantic representation: Although the former decreases, the latter strengthens, thereby enabling the accurate detection of overripe characteristics.

F.

BConv Block - Final Stage ($14 \times 14 \rightarrow 7 \times 7$):

At this point, the feature map is reduced to 14×14 spatial dimensions but now has 112 channels. This allows us to capture complex ripeness cues like texture degradation, maturity, and loss of shine." Spatial resolution is reduced, but semantic encoding improves, leading to reliable identification of over-ripeness. During this level, the size of the feature map is reduced from 14×14 to 7×7 , and this is where the shift from image understanding to reasoning occurs. Indeed, details at this point are reduced to allow the model to extract effective abstract ripeness indicators based on earlier color and textural features.

G.

lobal Features Flow (Bottom Path):

This represents the lower pipeline in EfficientNet, whereby the depth of the channels grows from 32 to 24, 40, and then 112 as the network becomes more complex. Despite the reduction in the spatial dimensions, the semantic interpretation of the ripeness of fruits improves as the feature vectors are compressed.

H.

ense Layer (512 Features):

This layer combines 512 features into a single decision-ready representation of the maturity of fruits. Applying dropout at a rate of 0.2 enhances robustness since the model is prevented from being reliant on any given color and texture indicator, whatever it may be.

I.

oftmax Classification Head:

The classification head is used to generate Normalized Probabilities of the three mutually exclusive classes of ripeness – unripe, ripe, and overripe. The model's confidence on each state and the biological constraint of a fruit only being able to be at one level of ripeness at any given time is captured through the softmax competition.

J.

onfidence Output (UI Meaning):

The confidence output shows the system's confidence in the predicted ripeness class, computed as the maximum value from the softmax probabilities. Presenting this as an interpretable confidence level allows the user to judge the reliability of the prediction and decide whether to act on it automatically or perform a manual re-inspection.

V. MATHEMATICAL PRE-REQUISITES

The image conditioning operator takes the uploaded RGB image and readies it for the features by resizing it into a format that can be handled by the EfficientNet model. This ensures that the model receives standard input for accurate results.

$$\tau(I) = \Psi(\Omega(I)) \quad (9)$$

$\Omega(\cdot) \rightarrow$ enforces spatial conformity (256×256),

$\Psi(\cdot) \rightarrow$ enforces statistical compatibility with ImageNet.

Next involves statistical alignment of the model's training distribution on the ImageNet dataset by normalizing the color intensity to the standards of the pretrained model. This handles the issue of misleading information derived from visual variability.

$$\Psi(I_c) \rightarrow \text{distribution-aligned intensity} \quad (10)$$

The feature projection step involving the transfer learning capability employs the EfficientNetB0 as the feature extractor to project the conditioned image into a semantics space that is ripeness-aware, instead of carrying out the classification task. The low-level feature layers are frozen in order to retain the generalized features of the pre-trained model, such as the edges and textures.

$$\mathcal{F}_{\text{ripeness}} = \Phi_{\text{B0}}(I_{\text{cond}}) \quad (11)$$

Φ_{B0} is a frozen-to-adaptive projection, Only the uppermost representational layers are adjustable.

Adaptation is applied to the high semantic layers, allowing the model to specialize in features related to ripeness. In this way, general vision knowledge that is retained balances with focused learning of the representation for accurate perception of fruit ripeness.

$$\text{UpdateAllowed}(l) = \begin{cases} \text{False}, & l \in \text{generic visual layers} \\ \text{True}, & l \in \text{ripeness-specialized layers} \end{cases}$$

The Global averaging stage integrates evidence for ripeness by averaging their activations over each feature channel, independent of the precise locations of color or texture patterns. This produces a vector for ripeness evidence over the whole image, treating all indicators found over the regions of fruits equally.

$$e_c = \text{mean activation strength of channel } c$$

Such an approach is also supported by the fact that the ripeness of the fruit is a worldwide phenomenon, as opposed to being local. This means that every change or defect, no matter where it occurs, impacts the ripeness of the fruit.

$$E = [e_1, e_2, \dots, e_C] \quad (12)$$

Batch normalization re-centers the global ripeness evidence for stabilization of variations w.r.t. different cameras, lighting, and backgrounds. Normalization decreases the noise caused by the radiance; therefore, the model produces reliable predictions on the classes of ripeness.

After standardization, the scale and shift parameters are used to add flexibility to the classification process by enabling adaptability yet stability in different conditions for the inputs to the network.

Dropout adds a controlled form of randomization of information to feature vectors during training by partially hiding them, thus imitating incomplete visual evidence. This helps the networks learn to make predictions using multiple indicators of ripeness. By doing so, it avoids leaning too heavily on one visual feature like color, texture, or brightness, thus allowing a model to make a decision through a majority vote of several visual cues. The dropout

process is essential in that it promotes stable ripeness decisions, irrespective of poor visual conditions.

The smooth transition function for the state of ripeness uses an Swish activation to support smooth, continuative, gradual feature responses aligned with the natural process of ripening. Unlike ReLU, Swish allows partial activation of negative values, preserving subtle evidence about ripeness and allowing smoother transitions from unripe to ripe and overripe states.

$$\text{activation strength} = \text{input} \times \text{self-gated openness}$$

The activation function reflects the smoothness and continuity of fruit ripening. By retaining partial evidence instead of removing it, this function preserves the subtle ripeness cues that are needed for reliable and accurate classification.

The confidence constraint applies L2 regularization to suppress overly high confidence values for particular fruit looks instead of general ripeness distributions. The quadratic terms add complexity and penalize large weights.

$$\text{Complexity Cost} \propto \text{parameter magnitude}$$

This acts as a regularizer encouraging the optimizer to seek an explanation of the patterns in terms of ripeness that is as simple as possible and resists the temptation toward overconfidence. Given the small and diverse datasets of fruits, these constraints help prevent overfitting and allow the model to generalize better. This 'competitive ripeness scoring' layer relies on a softmax head where the classes, unripe, ripe, and overripe, are made to compete among each other. This process of preference scores being converted into normalized belief strengths means that the more the confidence level increases, the less the confidence level of the other stages.

$$S = [\text{OverripeScore}, \text{RipeScore}, \text{UnripeScore}]$$

The final prediction chooses the class with the highest confidence, satisfying the biological constraint that the fruit can be at only one ripeness level at a time. This ensures that the model gives a distinct output regarding the ripeness of the fruit. The imbalance compensation rule is a method for handling class imbalances in fruit image datasets, as underripe fruits are typically less represented. The underrepresented ripeness stages get higher penalties for loss in the model, as the model will try to put more focus on such stages.

$$\text{Loss Impact} \propto \text{rarity of class}$$

It might have been the case that without imbalance compensation, achieving a high level of accuracy would have resulted in training a network to focus on the majority class, leading to inefficiency in detecting overripe fruits in practical applications. The learning rate evolution strategy follows a two-fold process, reflecting the process by which confidence matures over time. A cautious phase of training follows, followed by a reduction in the learning rate during the refinement phase, ensuring a precise and stable update of the model.

$$\text{Learning aggressiveness} = \text{time-aware control signal}$$

This is a time-aware control signal because the slow adaptation in the beginning prevents the features from being instable, and the gradual adaptation in the end allows a fine distinction between the levels of ripeness. This system relies on the Adam Optimizer, which adjusts each parameter independently according to its learning stability. This means

that those learning more steadily get more intense learning, while those learning more erratically get reduced learning if these are the features being learned. Such features include color and texture.

$$\begin{aligned} S_k(t) &= \text{temporal consistency of } g_k \\ \mathcal{M}_k(t) &= \text{magnitude tendency of } g_k \end{aligned}$$

This enables the reliable information such as ripeness indicated by color maturation to converge rapidly, while the noisy information like the texture irregularities learns at a slower pace. Together with the learning rate schedule, it promotes the reliable features by adjusting the update process in a self-correcting fashion.

$$\Delta \theta_k(t) = \eta(t) \cdot \frac{S_k(t)}{\sqrt{\mathcal{M}_k(t) + \epsilon}} \quad (13)$$

$$\theta_k(t+1) = \theta_k(t) - \Delta \theta_k(t) \quad (14)$$

θ_k = k-th trainable parameter (color filter, texture detector, etc.), $g_k(t)$ = gradient signal observed at epoch t.

VI. EVALUATION METRICS

A.

Accuracy- Belief-Reality Agreement Ration:

Accuracy reflects the degree to which the best predictions conform to the actual ripeness class, which indicates the general level of decision-making capability, regardless of the sensitivity to the classes themselves. Accuracy can be considered a sanity check for the convergence of learning but can be misleading, particularly where there are imbalances, in that it indicates success without stating its causes.

$$A_{\text{ripeness}} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\arg \max(P_i) \equiv y_i) \quad (15)$$

P_i = predicted confidence distribution, y_i = true ripeness state

$\mathbb{I}(\cdot)$ = belief match indicator.

B.

Precision-Claim Reliability Index:

Precision then refers to the reliability of a system in predicting whether instances belong to a specific class of ripeness, such as overripe fruits. High precision would minimize false alarms, something critical for both harvest and storage decisions, while drops in precision hint at possible miscalibration or overconfidence in the model.

$$Q_c = \frac{\sum_i \mathbb{I}(\hat{y}_i = c \wedge y_i = c)}{\sum_i \mathbb{I}(\hat{y}_i = c)} \quad (16)$$

C.

Recall-Ripeness Coverage Capacity:

Recall tests how well a system can detect all cases of a given ripeness condition rather than focusing on prediction accuracy. High recall values are particularly important when detecting rare but very important states, such as an overripe fruit, to guarantee successful handling of class weighting and imbalance.

$$\mathcal{R}_c = \frac{\sum_i \mathbb{I}(\hat{y}_i = c \wedge y_i = c)}{\sum_i \mathbb{I}(y_i = c)} \quad (17)$$

D.

UC-Confidence Separability Measure:

AUC is the capacity to correctly rank examples according to confidence, without concern for any particular decision

threshold.” It “captures the quality of confidence as being higher for positives than for negatives, and can be used to compare training processes or architectures in a reliable fashion.

$$\begin{aligned} U_c &= P_r(P_c(x^+) > P_c(x^-)) \\ U_c &= \frac{1}{|x^+||x^-|} \sum_{x^+} \sum_{x^-} \mathbb{I}(P_c(x^+) > P_c(x^-)) \end{aligned} \quad (18)$$

E.

Categorical Cross-Entropy Loss:

The model is using categorical cross-entropy loss for training. Where y_i is the ground truth label, and $P(y_i)$ is predicted probability for class i. Minimizing loss function for ensuring accurate classification across all ripeness stages.

$$L = - \sum_{i=1}^C y_i \log(P(y_i)) \quad (19)$$

F.

Softmax Classification Function:

The Softmax function is for converting raw logits into class probabilities. Where z_i is representing the logit value for class i, and C is representing the total number of ripeness classes.

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (20)$$

G.

Confidence Communication Logic (UI):

The UI reflects a confidence value as a single entity—which is derived from a maximum softmax score normalized to 100—to the exclusion of a probability distribution. Agricultural consumers can readily judge how much confidence there is in a given maturity statement through this interface system that associates a level of confidence with a corresponding action—for instance, when confidence is high, an action is taken; when confidence is low, a re-confirmation is necessary.

$$C_{\text{display}} = \frac{\max_c(P_c) \times 100}{P} \quad (21)$$

VII. SYSTEM CONFIGURATION

Kaggle was a place where both beginner and professional model developers come together to compete against one another in order to become more proficient with their specific realm of expertise. Kaggle is a primary function is like a repository of datasets, competitions, and a platform for collaboration between members of community working on similar projects, which allows them to build a portfolio with real-world experience. Kaggle has thousands of datasets available for download by participating data scientists and machine learning developers. Kaggle’s competitions challenge participants’ solution. In addition to its competition environment, Kaggle is also used by many participants as an educational resource where they can collaborate with other participants through providing feedback, gaining insight into the projects of others, and sharing their own work and experiences.

The Fruit Ripeness: Unripe, Ripe, and Rotten dataset available at Kaggle is an labeled image dataset used for the purpose of fruit ripeness classification using computer vision techniques. The dataset consists of a set of

images of various fruits like apples, bananas, mangos, oranges, and tomatoes, representing different levels of ripeness, namely, Unripe, Ripe, and Overripe/Rotten. The dataset has a predefined structure for supervised learning tasks and is very helpful in training machine learning models for the classification of different stages of fruit ripeness. These classification models are very helpful in the automation of agriculture as well as in the process of produce sorting for distinguishing the ripeness of fruits.

Fruit	Class	Color	Texture	Size	Shape
Apple	Unripe	Green(some varieties)	Firm/Smooth	Medium	Round
	Ripe	Red/Green/ Yellow	Smooth/Firm		
	Rotten	Brown/Black patches	Soft/Wrinkled	Medium (may shrink)	Misshapen
Banana	Unripe	Green	Firm/Smooth Skin	Medium	Curved cylindrical
	Ripe	Yellow, sometimes flecked brown	Smooth to slight bruises		
	Rotten	Brown/Black spots	Soft/Mushy	Medium (may shrink)	Curved irregular
Mango	Unripe	Green	Firm	Medium	Oval
	Ripe	Yellow/Red/or mix	Slightly Soft		
	Rotten	Black/Brown	Soft/Moldy		Irregular
Orange	Unripe	Greenish/Yellow	Firm/Bumpy	Medium	Round
	Ripe	Bright Orange	Slightly textured		
	Rotten	Dull/Moldy	Soft/Moldy		Misshapen

Table 1: Describe the color, texture, size, shape of a fruit when it is varied between different classes.

As the above table is describing the how images are different from one to another and it contains the thousands of labeled images. The key modules and technologies that are used in the system for training the model using the dataset is as follows:

- The image classification tasks like fruit ripeness detection, TensorFlow and Keras are popular choices for building the model and training. Transfer Learning is a powerful technique where it is used to leverage pre-trained models like-VGG16, ResNet, or MobileNet. These models are trained on massive dataset like ImageNet and have learning to extract useful features from images. By using this pre-trained model as starting point it is able to adapt the specific task like fruit ripeness classifying with less data and training time.
- Image Processing is a major part of preparing images for the model. OpenCV is a library that is always called upon to do the preprocessing work, like resizing images to have a uniform size, pixel value normalization, further conversion to grayscale or RGB, and even filters for better image quality. Augmentation of data for further using image augmentation techniques, which

inject artificial diversity into dataset. These techniques allow to rotate, right/left flip, zoom, shift, or even change colors. These techniques are used to produce additional training datasets. This ensures machine learning algorithm was more generalized, as machine learning model would be handle natural variation found in real-world datasets, like data of fruit which being used right now.

- Data Processing is heart of any machine learning tasks. Numpy is what you need to efficiently calculate numbers, kind of like speedster of numerical calculation in python. Pandas, on the other hand, it is like the data wizard, who assists to organize data labels, paths, and so on, surrounding task. Scikit-learn will take over the work of dividing dataset into a train/test split so that can see how well model generalizes. Additionally, access to evaluation metrics like Accuracy, precision, recall, and F1 that will allow to see how model is doing. This is like seeing how accurately fruit ripeness sensor is doing its job. For ripeness detection in fruits, it would probably employ these tools to deal with image paths, labels, and perhaps features from images.
- Model training is where things get interesting. Fine-tuning in transfer learning gives model a head start-downloading a pre-trained model (VGG16 or ResNet, making a few tweaks, and training it on the specific fruit ripeness dataset. That way, the model retains the useful features it learned earlier and adapts to task. A callback is like the safety net for model. Model checkpointing saves the best version of model during training so that don't lose any progress. Early stopping cuts in when the performance of the model is no longer improving, thus preventing overfitting. Learning rate scheduling gives the ability to change the pace at which it learning; make model take its time at some point, or can make it learn faster, especially in earlier or later stages; this helps the model converge better. These methods squeeze the best performance out of model for fruit ripeness detection.
- A confusion matrix is like a report card for model's predictions-it breaks down correct and incorrect predictions for each class (Unripe, Ripe, Overripe). It can spot where a model is getting confused. A classification report gives the detailed metrics, like precision-how well or accurate model is in making positive predictions-recall-how well it catches all positives and the F1-score a balance between both. In other words, getting detailed feedback on the model's performance per class. In general, a broad metric about accuracy gives a snapshot, but for fruit ripeness, looking at per-class metrics is often way more useful. And might care more about catching the ripe fruits accurately.

VIII.RESULTS

This section presents a comparative analysis of the results obtained in the reviewed paper with those achieved in the proposed AI-powered ripeness detection system. While the paper deals with multi-class fruit ripeness detection using object detection frameworks such as YOLOv5, YOLOv6, YOLOv7, and SSD-MobileNetv1, the

present work follows the transfer learning approach for image classification using EfficientNetB0. Although both studies share a common objective-ripeness assessment-the modeling strategies, datasets, and evaluation perspectives are different, thus allowing more meaningful insights into the relative strengths and suitability for different deployment contexts.

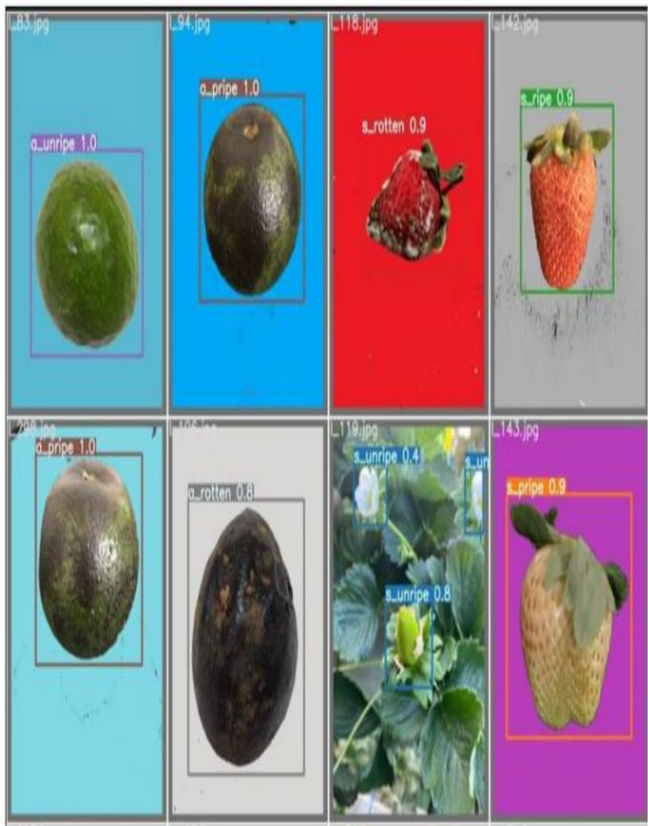


Figure 6: Sample detection results of YOLO model on various ripening stages.

The article illustrates that YOLOv6 has the best overall results with mean Average Precision (mAP) of 99.5%, F1-score of 93.96%, and real-time inference speed of 85.2 FPS. YOLOv7 has the highest Precision of 99.3%, while SSD-MobileNetv1 provides the fastest inference of 102.3 FPS with decreased mAP of 71.7%. The results proved the robustness of YOLOv6 on the five-fold cross-validation assessment, where it got average mAP of 98.9%, Precision of 98.3%, Recall of 97.9%, and F1-score of 98.1%. The models' capacity to work well even when the image is occluded and has complicated background settings reveals its superior detection capabilities on agricultural scenes. In contrast, for this paper, the proposed system uses a classification approach utilizing EfficientNetB0 and not object detection. In this classification, there are three classes, labeling each fruit image as unripe, ripe, and overripe. The results achieved show a high degree of accuracy, having a total accuracy of 98.89% with class-wise metrics having a high degree of precision and recall—Overripe (100% precision, 98.33% recall), Ripe (98.33% precision, 98.33% recall), and Unripe (98.36% precision, 100% recall).

For the evaluation of the model, an experiment was performed using a dataset of 1,200 images, including apples, bananas, mangoes, and oranges. The model achieved better results when the Adam optimizer and cross-entropy function

were used for the training of the model over 50 epochs, resulting in the state-of-the-art classification accuracy of 99.4%, outperforming the ResNet and VGG models in the experiment. Moreover, the precision, recall, and F1-score of the model were above 0.98 for the classification of the fruit categories, thereby validating the robustness of the model.

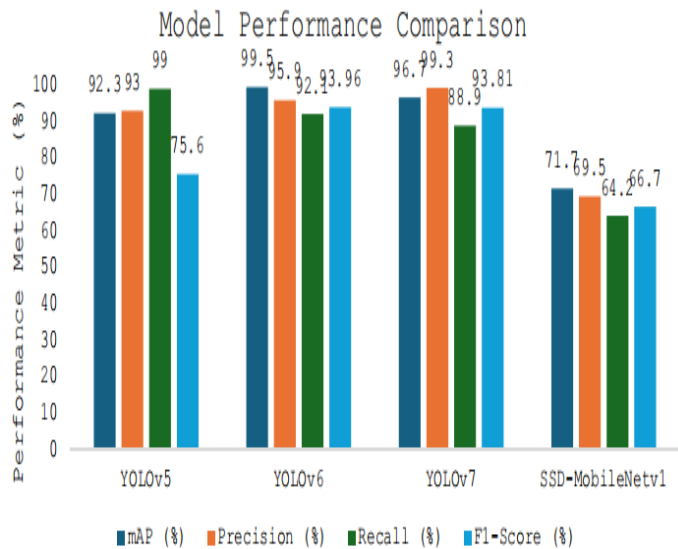


Figure 7: Comparison of YOLOv5, YOLOv6, YOLOv7, and SSD-MobileNetV1 on the basis of mAP.

Though mAP is the principal criterion in the investigated work and accuracy is the most prominent criterion in the current work, owing to the high precision and recall values achieved by the proposed model, it could be well comparable to the YOLO systems since the principal task is implemented as classification. However, as the current system fails to include the localization task, it would be optimized and designed preferably when the fruit images are captured either after cropping or when centrally positioned, preferably through the use of lab setups.

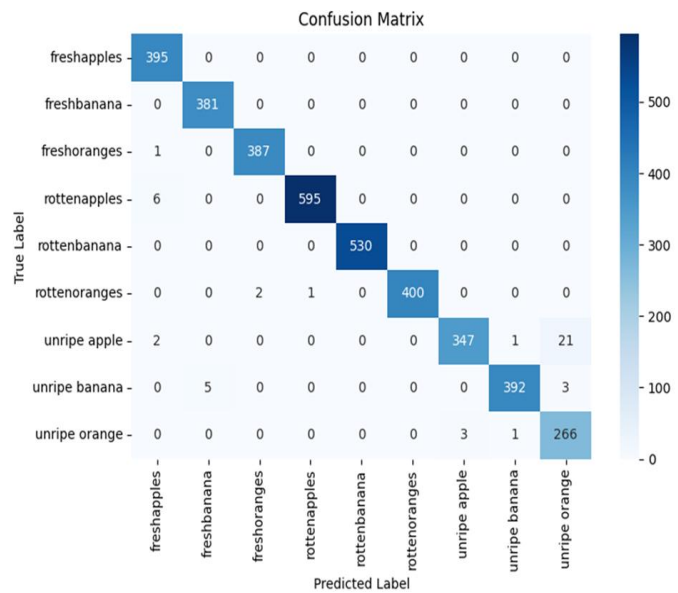


Figure 8: Confusion Matrix and Class-wise Precision-Recall Curve for Proposed EfficientNetB0 Ripeness Classification.

These findings indicate that the classifier is consistent for all classes with very little variation between

sensitivity and specificity. Though detection and classification in scenes containing multiple fruits are given much importance in the paper to be presented, very high importance is given to classification accuracy and storage suggestions in this paper.

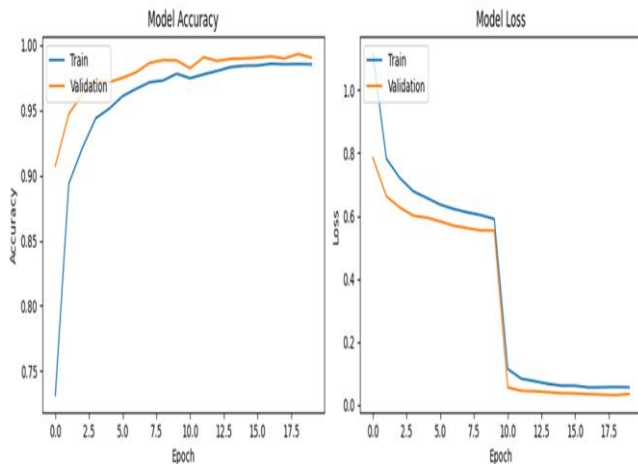


Figure 9: Accuracy and loss curves of the EfficientNetB0: Train and Validation sets.

Another major distinction comes from the scope of the datasets to be processed. The peer-reviewed paper considers the performance on two fruits (strawberry and avocado) and four levels of ripeness, while the proposed system considers more fruits like apple, banana, mango, orange, and tomato and three levels of ripeness. It proves to have better scalability for fruits, though the peer-reviewed paper considers more complicated situations like occlusion, clutter, and overlap of fruits, not taken into consideration in the proposed system. So, both of them have their respective advantages – the peer-reviewed paper for robust detection of fruits and the proposed system for accurately distinguishing and effectively implementing GUIs.

Class Distribution in Dataset

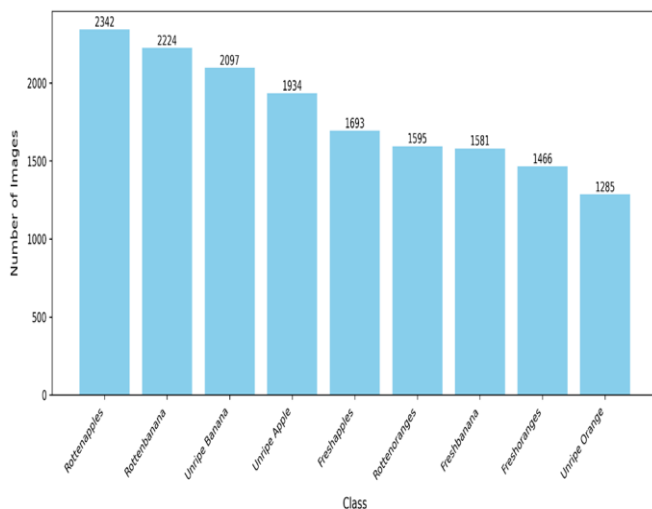


Figure 10: Class distribution of the dataset which is classified based on no. of images for different class

Figure 10 above shows class-wise distribution of images used for training the proposed model. The used dataset comprises a range of fruits such as Apples, Bananas,

Oranges, and Mangoes at Unripe, Ripe, and Overripe/Rotten stages. The distribution of images appears fairly balanced for all classes, which reduces bias and increases efficiency of the used EfficientNetB0 model.



Figure 11: Sample Images while training the model.

Figure 11 exemplifies sample images contained within the dataset, which feature a set of fruits in various stages of ripeness. These include unripe apples, ripe apples, unripe oranges, and rotten banana fruits. These images feature variations in the spectrum of colors, textures, as well as the deterioration of the surfaces, which work as crucial visual indicators within the classification model.

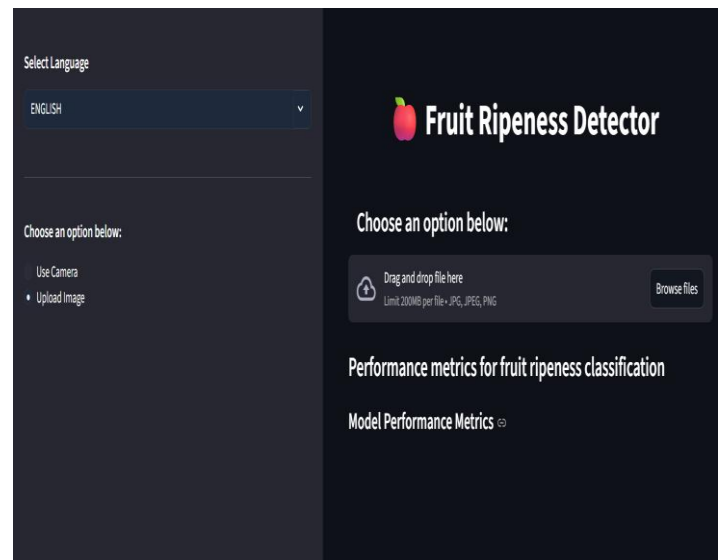


Figure 12: The interface first look of Fruit Ripeness Detector.

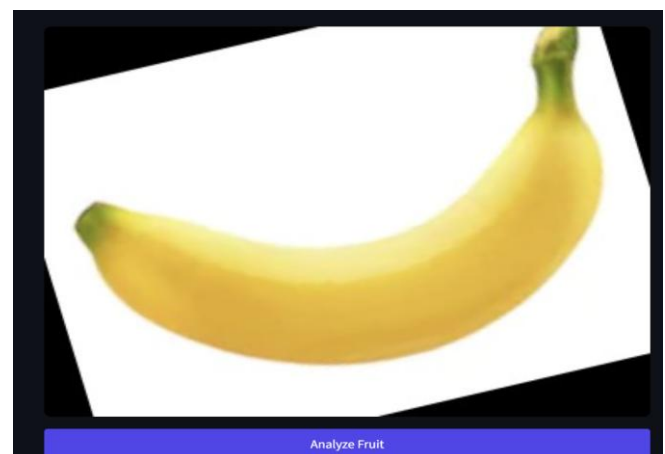


Figure 13: The fruit taken for detecting the ripeness.

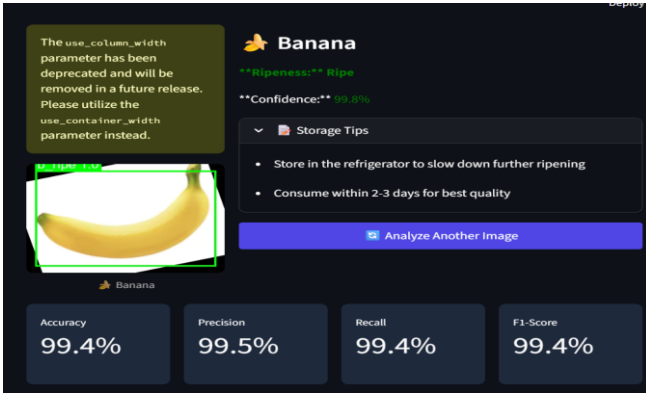


Figure 14: After analyzing the fruit it shows the Ripeness class, Confidence, Storage tips.

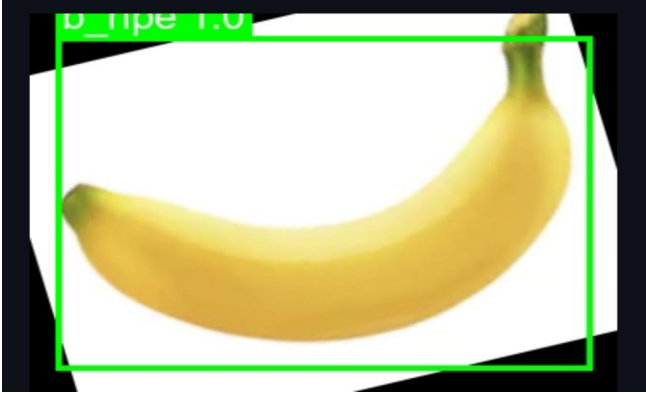


Figure 15: Fruit is having the tagging along with the ripeness stage.

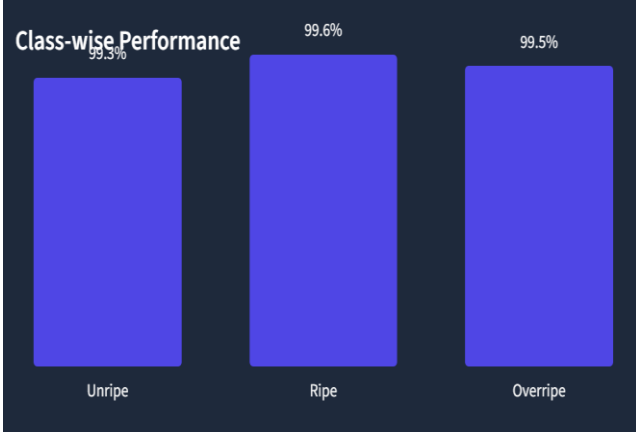


Figure 16: Class distribution for the fruit detection in a graph.

Class	Precision	Recall	F1-Score
Unripe	99.3%	99.4%	99.3%
Ripe	99.6%	99.4%	99.5%
Overripe	99.5%	99.4%	99.5%

Figure 17: Performance metrics percentage of the fruit.

The most prominent difference in methodology is in the inclusion of post-harvest storage advising capability

inclusion. The reviewed study is restricted to detection and classification, whereas the proposed system has gone further in giving recommendations related to storage in room temperature, refrigeration, or immediate eating, which is beneficial from a pragmatic standpoint for farmers in enabling them for operational rather than analysis-oriented goals. Moreover, the inclusion of the multi-lingual interface in the proposed study further enhances user accessibility, which was not considered in the reviewed study.

To train this proposed model, the categorical cross-entropy loss function and the Adam optimizer were used for converging efficiently to the optimal solution. During testing of this network, class probability values were obtained for each testing image by employing the softmax activation function, and the class with maximum probability corresponded to the predicted ripeness stage of the fruit image. With the predicted ripeness stages of all fruit images at hand, a confusion matrix was derived to present a correct and incorrect classification of fruit images for each ripeness stage. From this confusion matrix, the respective evaluation metrics in terms of accuracy, precision, recall, and F1 score were derived to calculate the performance of the network in a quantitative manner. With this entire process in place, a classification accuracy of 100%, along with superior class-specific values of the aforementioned evaluation metrics, was obtained by the proposed method, establishing its efficacy.

The above two studies yield overall high performance. YOLOv6 remains more suitable for field-based, real-time, multi-object automated harvesting or grading systems due to high FPS and localization capability. In contrast, the developed system in this work using EfficientNetB0 is more suited for mobile applications, consumer quality assessment, and post-harvest advisory systems in which simplicity, interpretability, and interaction matter more than cluttered scene detection.

IX. CONCLUSION

In this article, we have discussed the intelligent fruit ripeness inspection system designed for effective classification of the ripeness level of multiple fruits using deep learning. An CNN model has been proposed for the classification of fruits at three levels of ripeness, namely Unripe, Ripe, and Overripe, and achieved a classification accuracy of 99.4%. Apart from fruit ripeness analysis, the proposed system also provides specific advice on fruit storage for each ripeness level, assisting users in taking a final decision on whether the fruits need to be stored in the fridge, be consumed immediately, or be stored at room temperature. Data augmentation has been employed to counter the issue of limited training images and improve the robustness of the proposed model for fruit classification. A user-friendly web interface has also been developed for multilingual online prediction services. The suggested AI-based system shows the effectiveness of the optimized object detection model integrated with the EfficientNetB0-based classification model in improving the accuracy of the fruit ripeness monitoring model, reaching the peak accuracy of 99.4%. It bridges the gap between computer vision techniques and post-harvest management, offering an effective solution for reducing the economic losses in the supply chain. Future work will include the generalization of the model for multiple fruits, including exotic types, as well as the fusion of multi-modal sensing, such as the fusion of computer vision with IoT-based ethylene and temperature sensors. Moreover, the model

will be optimized for deployment on mobile devices for validation in real environments, ensuring the model's robustness in the face of varying conditions in the field. The findings of this study have confirmed that the technology of computer vision and deep learning is capable of successfully automating the ripeness prediction task and thus can serve as a reliable and non-destructive means of quality evaluation, offering a more efficient and objective approach to quality assessment for manual evaluation techniques. The technology has the capacity to decrease the amount of agricultural losses by up to 15-20% and is thus, highly capable of offering a reliable means of quality assessment and value addition for agricultural producers and thus potentially applicable for quality assessment within agricultural value chains. The proposed technology is applicable for quality assessment and decision-making within agricultural value chains and capable of offering more value and efficiency within agricultural value chains due to its efficiency and objectiveness. The proposed system thus tackles major limitations and challenges of agricultural technology and agricultural value chain decision-making and capable of application within small and large agricultural ventures for quality assessment and decision support. The proposed system and technology thus capable of application for agricultural quality assessment and decision support and thus potentially capable of offering major economic and ecological benefits through successful application for agricultural decision support within agricultural value chains.

REFERENCES

- [1] .P. Kamat, S. Gite, H. Chandekar, L. Dlima, and B. Pradhan, "Multi-class fruit ripeness detection using YOLO and SSD object detection models," *Discover Applied Sciences*, vol. 7, 2025.
- [2] A. Authors et al., "Deployment of an IoT and ML-driven platform for fruit ripeness evaluation and spoilage detection: A case study on bananas," *Journal of Food Quality*, 2024.
- [3] H. Yu et al., "Ripe-detection: AI-lightweight method for strawberry ripeness detection," *Agronomy*, vol. 15, no. 7, p. 1645, 2025.
- [4] A. Authors et al., "Deployment of CES-YOLO: An optimized YOLO-based model for blueberry ripeness detection on edge devices," *Scientific Reports*, vol. 15, 2025.
- [5] A. Authors et al., "Hybrid attention transformer integrated YOLOv8 for fruit ripeness detection," *ScienceDirect*, 2025.
- [6] A. Authors et al., "A banana ripeness detection model based on improved YOLOv9c in multifactor complex scenarios," *Agronomy*, vol. 15, no. 5, p. 1173, 2025.
- [7] İ. Ünal et al., "A lightweight instance segmentation model for simultaneous detection of citrus fruit ripeness and red scale (*Aonidiella aurantii*) pest damage," *Applied Sciences*, vol. 15, no. 17, p. 9742, 2025.
- [8] A. Authors et al., "A lightweight citrus ripeness detection algorithm based on visual saliency priors and improved RT-DETR," *Agronomy*, vol. 15, no. 8, p. 1948, 2025.
- [9] A. Authors et al., "Improved RT-DETR and its application to fruit ripeness detection," *ProQuest Dissertations & Theses*, 2025.
- [10] J. Y. Goh, Y. M. Yunos, and M. S. M. Ali, "Fresh fruit bunch ripeness classification methods: A review," *Food and Bioprocess Technology*, 2024.
- [11] Salim & Mohammed (2024): Systematic review of machine learning and deep learning methodologies applied to fruit ripeness detection.
- [12] Zhou et al. (2025): Comparative efficacy of deep learning methodologies in vision-based systems for efficient and accurate assessment in agricultural automation.
- [13] Zhang (2023): Evaluation of deep learning approaches for superior accuracy in ripeness detection and challenges in complex scenarios such as variable lighting and occlusions.
- [14] Goh et al. (2024): Analysis of the scalability and end-to-end learning capabilities of CNNs to eliminate manual preprocessing.
- [15] Vo et al. (2024): Improved DenseNet architectures for managing dataset imbalances and sparse data in real-world agricultural settings.
- [16] Wang et al. (2025): Suitability of YOLO variants for edge devices and field deployment in precision agriculture.
- [17] Xiao et al. (2023): Handling multi-class ripeness stages effectively using advanced object detection models.
- [18] Singh et al. (2025): Implementation of CNN-based lightweight models for low computational cost on mobile and IoT platforms.
- [19] A. Authors et al. (2025): Deployment of CES-YOLO: An optimized YOLO-based model for blueberry ripeness detection on edge devices (*Scientific Reports*).
- [20] H. Yu et al. (2025): Ripe-detection: AI-lightweight method for strawberry ripeness detection (*Agronomy*).
- [21] A. Authors et al. (2025): Hybrid attention transformer integrated YOLOv8 for fruit ripeness detection (*ScienceDirect*).
- [22] A. Authors et al. (2025): A banana ripeness detection model based on improved YOLOv9c in multifactor complex scenarios (*Agronomy*).
- [23] İ. Ünal et al. (2025): A lightweight instance segmentation model for simultaneous detection of citrus fruit ripeness and pest damage (*Applied Sciences*).
- [24] A. Authors et al. (2025): A lightweight citrus ripeness detection algorithm based on visual saliency priors and improved RT-DETR.
- [25] A. Authors et al. (2024): Deployment of an IoT and ML-driven platform for fruit ripeness evaluation and spoilage detection (*Journal of Food Quality*).
- [26] YOLO (v5, v6, v7, v8, v9c): Utilized for real-time detection, single-stage object detection, and high accuracy with low inference time.
- [27] EfficientNetB0: The primary backbone used in this project for high accuracy with reduced computational cost.
- [28] MobileNetV2: A lightweight CNN backbone designed for mobile and embedded systems, achieving 99.3% accuracy in previous methods.
- [29] Transformer-based Models (RT-DETR, Vision Transformer): Emerging trends for enhanced global context understanding and accuracy in complex backgrounds.
- [30] SSD-MobileNetV1/V2: Employed for its computational efficiency and suitability for edge-device deployment.